

PPO Reinforcement Learning Design Documentation for Energy Harvesting Devices

Department of Electrical and Computer Engineering
Graduation Project Documentation

November 3, 2025

Contents

1	System Overview	2
1.1	Basic Idea	2
2	Simulator Environment Design	2
2.1	Environment Variables	2
2.2	State Representation	2
2.3	Action Space	3
3	Reward Function Design	3
3.1	Overall Structure	3
3.2	Quality Reward	3
3.3	Energy Reward	3
3.4	Buffer Penalty	3
3.5	Switching Penalty	4
3.6	Shutdown Penalty	4
3.7	Parameter Summary	4
3.8	Interpretation	4
4	PPO Agent Design	5
4.1	Algorithm Overview	5
4.2	Network Architecture	5
4.3	PPO Configuration (Simulation)	5
5	Simulation Flow	5
6	Computational Efficiency Considerations	6
7	Planned Experiments	6
8	Future Work	6

1 System Overview

The goal of this project is to design an adaptive reinforcement learning (RL) controller that optimizes the performance of an IoT system powered by solar energy harvesting. The main objectives are to:

- Increase processing quality whenever possible.
- Prevent system shutdown due to energy depletion.
- Avoid buffer overflow and data loss.

1.1 Basic Idea

At each timestep, the agent observes system variables such as voltage, buffer level, and irradiance. Based on these inputs, the PPO policy selects one of three actions:

1. **High-Quality Model** – high accuracy, high energy cost.
2. **Degraded Model** – moderate accuracy, lower energy cost.
3. **Sleep Mode** – checkpoint state and conserve energy.

The general decision rules are as follows:

- If sufficient energy and memory are available $\rightarrow$ select *High Quality*.
- If high-quality processing risks shutdown or overflow $\rightarrow$ select *Degraded*.
- If even degraded mode risks shutdown $\rightarrow$ enter *Sleep*.

2 Simulator Environment Design

The RL environment simulates the coupled energy and data dynamics of a solar-powered embedded device. It is designed for stable PPO training and later transfer to a real-world energy-harvesting platform.

2.1 Environment Variables

- $V(t)$ – supercapacitor voltage.
- $B(t)$ – buffer occupancy (number of frames).
- $G(t)$ – solar irradiance (harvested energy input).
- a_{t-1} – previous agent action.

2.2 State Representation

The state vector at time t is given by:

$$s_t = [v_{\text{norm}}, b_{\text{norm}}, g_{\text{norm}}, \Delta g, t_{\text{sin}}, t_{\text{cos}}, a_{t-1}]$$

Each term is normalized to the range $[0, 1]$ for stable PPO training. Normalization constants are computed once per episode, minimizing per-step floating-point operations and improving computational efficiency.

2.3 Action Space

$$\mathcal{A} = \{0, 1, 2\} = \{\text{High Quality}, \text{Degraded}, \text{Sleep}\}$$

3 Reward Function Design

The reward function defines how the PPO agent learns to balance energy efficiency and processing quality. It combines several sub-rewards representing different objectives of the system.

3.1 Overall Structure

The total reward at each timestep t is:

$$r_t = r_{\text{quality}} + r_{\text{energy}} + r_{\text{buffer}} + r_{\text{switch}} + r_{\text{shutdown}} \quad (1)$$

Each component is defined below.

3.2 Quality Reward

$$r_{\text{quality}} = w_h n_h + w_d n_d \quad (2)$$

where:

- n_h = number of frames processed using the **High-Quality** model.
- n_d = number of frames processed using the **Degraded** model.
- w_h, w_d = reward weights ($w_h > w_d$).

Intuition: Encourages high-quality processing when energy allows.

3.3 Energy Reward

$$r_{\text{energy}} = w_e \cdot \frac{V(t) - V_{\min}}{V_{\max} - V_{\min}} \quad (3)$$

where:

- $V(t)$ = current capacitor voltage.
- $V_{\min}, V_{\max}$ = voltage bounds.
- w_e = weight controlling the energy incentive.

Intuition: Rewards maintaining voltage safely above $V_{\min}$.

3.4 Buffer Penalty

$$r_{\text{buffer}} = -w_b \left(\frac{B(t)}{B_{\max}} \right)^2 \quad (4)$$

where:

- $B(t)$ = current buffer occupancy.
- $B_{\max}$ = buffer capacity.
- w_b = penalty weight.

Intuition: Strongly discourages high buffer occupancy to avoid overflow.

3.5 Switching Penalty

$$r_{\text{switch}} = -w_s \cdot \mathbb{K}_{(a_t \neq a_{t-1})} \quad (5)$$

where:

- a_t, a_{t-1} = current and previous actions.
- $w_s = 0.01$ = penalty coefficient.

Intuition: Reduces frequent action switching to improve stability.

3.6 Shutdown Penalty

$$r_{\text{shutdown}} = -R_{\text{shut}} \cdot \mathbb{K}_{(V(t) < V_{\min})} \quad (6)$$

where:

- R_{shut} = large penalty constant (e.g., 50).

Intuition: Strongly penalizes critical undervoltage events to prevent system shutdown.

3.7 Parameter Summary

Table 1: Reward Function Parameters

Symbol	Meaning	Example Value	Unit	Purpose
w_h	High-quality reward weight	1.0	–	Reward for accurate inference
w_d	Degraded-quality reward weight	0.6	–	Reward for reduced-quality inference
w_e	Energy reward weight	0.05	–	Incentivizes safe energy levels
w_b	Buffer penalty weight	5.0	–	Prevents overflow
w_s	Switching penalty weight	0.01	–	Reduces oscillations
R_{shut}	Shutdown penalty	50.0	–	Penalizes shutdown
$V_{\min}$	Minimum voltage	2.6	V	Shutdown threshold
$V_{\max}$	Maximum voltage	5.5	V	Max capacitor voltage
$B_{\max}$	Buffer capacity	10	frames	Maximum buffer size

3.8 Interpretation

This reward formulation ensures:

- Positive rewards for inference tasks, especially high-quality ones.
- Incentives for maintaining safe energy margins.
- Strong penalties for buffer overflow and undervoltage events.
- Mild penalties for unnecessary mode switching.

Overall, this promotes long-term reliability, sustained operation, and adaptive performance under variable solar conditions.

4 PPO Agent Design

4.1 Algorithm Overview

Proximal Policy Optimization (PPO) is a policy gradient method that uses a clipped surrogate objective to ensure stable updates and prevent destructive policy shifts.

- **Policy:** $\pi_\theta(a|s)$ outputs action probabilities.
- **Value Function:** $V_\phi(s)$ estimates expected returns.

4.2 Network Architecture

Both policy and value networks share the same structure:

- Input: 7 state features.
- Hidden Layers: Two fully connected layers with 64 neurons each.
- Activation: ReLU.
- Output: Three neurons (one per action) with Softmax activation.

4.3 PPO Configuration (Simulation)

```
PPO(  
    policy = "MlpPolicy",  
    env = EnergyHarvestEnv(),  
    n_steps = 2048,  
    batch_size = 256,  
    n_epochs = 10,  
    gamma = 0.99,  
    gae_lambda = 0.95,  
    clip_range = 0.2,  
    learning_rate = 3e-4,  
    ent_coef = 0.01  
)
```

5 Simulation Flow

The PPO agent interacts with the simulator as follows:

1. Initialize voltage, buffer, and irradiance values.
2. PPO policy selects action a_t based on state s_t .
3. Environment updates:
 - Update capacitor energy balance.
 - Update buffer occupancy.
 - Compute next irradiance and input energy.
4. Compute reward r_t and next state s_{t+1} .
5. PPO collects experiences and updates the policy periodically.

6 Computational Efficiency Considerations

Although this phase is simulation-only, the PPO design is optimized for eventual deployment on low-power embedded hardware.

- **Reduced Floating-Point Use:** Normalization constants precomputed per episode.
- **Compact State Vector:** Only seven essential features included.
- **Lightweight Neural Network:** Two small hidden layers balance accuracy and speed.
- **Quantization-Ready:** Architecture suitable for int8 quantization on microcontrollers.

These choices ensure compatibility with TensorFlow Lite Micro or similar embedded inference runtimes.

7 Planned Experiments

Planned simulation experiments include:

- Comparing PPO performance with rule-based baselines (voltage thresholds).
- Testing policy adaptability under varying irradiance profiles.
- Measuring shutdown and overflow frequencies.
- Evaluating trade-offs between average inference quality and system uptime.

8 Future Work

Future extensions include:

1. Integrate real-world irradiance data (NREL dataset).
2. Quantize the PPO policy to int8 for deployment.
3. Deploy on microcontroller or low-power SoC hardware.
4. Evaluate real-time energy-performance trade-offs.